

1. Quick Sort

Aim:

To write a C program to implement Quick Sort and analyze its time complexities.

Description :

Quick Sort is a sorting algorithm based on the Divide and Conquer technique. It works by selecting an element called the pivot and partitioning the array around the pivot such that the pivot is placed in its correct sorted position.

In this method, an element is chosen as the pivot. The array is then rearranged so that:

- All elements smaller than the pivot are placed on the left side.
- All elements greater than the pivot are placed on the right side.

After placing the pivot in its correct position, the same process is recursively applied to the left and right subarrays until the entire array becomes sorted.

Quick Sort performs sorting in-place, meaning it does not require additional memory for merging like Merge Sort. Partitioning is the core operation of Quick Sort.

Working Principle :

1. Choose a pivot element from the array.
2. Partition the array around the pivot.
3. Place the pivot in its correct sorted position.
4. Recursively apply Quick Sort on the left subarray.
5. Recursively apply Quick Sort on the right subarray.

Example :

Consider the array:

[25, 10, 35, 15, 5, 30, 20]

If 20 is chosen as the pivot, after partitioning the array becomes:

[10, 15, 5, 20, 35, 30, 25]

Now,

Left Subarray : [10, 15, 5]

Right Subarray : [35, 30, 25]

After recursively applying Quick Sort to both subarrays, the final sorted array is:

[5, 10, 15, 20, 25, 30, 35]

Thus, the array is completely sorted.

Time Complexity Analysis of Quick Sort :

Best Case: $O(n \log n)$

Occurs when the pivot divides the array into two equal halves.

Average Case : $O(n \log n)$

Occurs when the pivot divides the array into reasonably balanced parts.

Worst Case : $O(n^2)$

Occurs when the smallest or largest element is always chosen as the pivot.

Auxiliary Space :

Average Case : $O(\log n)$ (due to recursive call stack)

Worst Case : $O(n)$

Program:

```
#include <stdio.h>

void swap(int* a,
          int* b) { int temp
                  = *a;
                  *a = *b;
                  *b = temp;
}

int partition(int arr[], int low,
              int high) { int pivot =
                          arr[high];
                          int i = (low - 1);
                          for (int j = low; j <
                              high; j++) { if
                              (arr[j] < pivot) {
                                  i++;
                                  swap(&arr[i], &arr[j]);
                              }
                          }
                          swap(&arr[i + 1],
                              &arr[high]); return (i
                              + 1);
}

void quickSort(int arr[], int low, int
               high) { if (low < high) {
    int pi = partition(arr, low,
                       high); quickSort(arr, low,
```

```
        pi - 1); quickSort(arr, pi +  
        1, high);  
    }  
}
```

```
void printArray(int arr[], int size)
```

```
    for (int i = 0; i < size;  
i++) printf("%d ", arr[i]);  
        printf("\n");  
}
```

```
int main() {  
    int arr[] = {10, 7, 8, 9, 1, 5};  
    int n = sizeof(arr) /  
    sizeof(arr[0]);  
    printf("Unsorted array: \n");  
    printArray(arr, n);
```

```
    quickSort(arr, 0, n - 1);
```

```
    printf("Sorted  
array: \n");  
    printArray(arr,  
n);  
    return 0;  
}
```

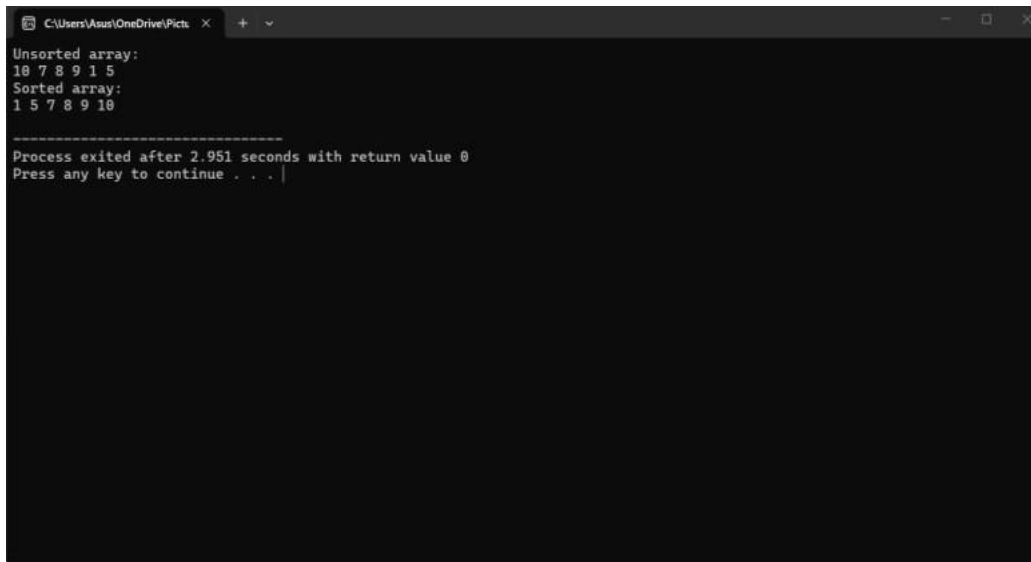
Output:

Unsorted array:

10 7 8 9 1 5

Sorted array:

1 5 7 8 9 10



```
C:\Users\Aoun\OneDrive\Pict... X + -
Unsorted array:
10 7 8 9 1 5
Sorted array:
1 5 7 8 9 10

=====
Process exited after 2.951 seconds with return value 0
Press any key to continue . . . |
```

Result: Hence the program of quick sort has executed and its output verified successfully.

2. Merge Sort

Aim:

To write a C program to implement Merge Sort and analyze its time complexities.

Description :

Merge Sort is a sorting algorithm that follows the Divide and Conquer approach. It divides the given array into smaller subarrays, sorts each subarray recursively, and then merges the sorted subarrays to produce the final sorted array.

Unlike simple sorting methods such as Bubble Sort or Selection Sort, Merge Sort does not repeatedly compare adjacent elements. Instead, it divides the problem into smaller parts, solves each part independently, and then combines the results in sorted order.

Merge Sort is a stable sorting algorithm, meaning it maintains the relative order of equal elements. However, it requires additional memory for merging the subarrays.

Working Principle :

Merge Sort works in three main steps:

1. Divide :

Divide the array into two halves recursively until each subarray contains only one element.

2. Conquer :

Each subarray is considered sorted because it contains only one element.

3. Merge :

Merge the sorted subarrays back together in sorted order.

The process continues until all elements are merged into a single sorted array.

Example :

Consider the array:

[38, 27, 43, 3, 9, 82, 10]

After dividing the array repeatedly:

Left Half : [38, 27, 43]

Right Half : [3, 9, 82, 10]

After sorting and merging the subarrays, the final sorted array becomes:

[3, 9, 10, 27, 38, 43, 82]

Thus, the entire array is sorted using the divide and merge process.

Time Complexity Analysis of Merge Sort :

Best Case : $O(n \log n)$

Average Case : $O(n \log n)$

Worst Case : $O(n \log n)$

Merge Sort has the same time complexity in all cases because it always divides and merges the array in the same way.

Space Complexity :

Auxiliary Space : $O(n)$

Extra memory is required to store temporary arrays during merging.

Program:

```
#include <stdio.h>

void merge(int arr[], int l, int m, int r) { int
    n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) { if
        (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        } k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
    }
```

```
    k++;  
}
```

```
while (j < n2) {  
    arr[k] = R[j];  
    j++;  
    k++;  
}  
}
```

```
void mergeSort(int arr[], int l, int r) { if  
    (l < r) {  
        int m = l + (r - l) / 2;  
  
        mergeSort(arr, l, m); mergeSort(arr, m  
        + 1, r);  
  
        merge(arr, l, m, r);  
    }  
}
```

```
void printArray(int arr[], int size) { for  
    (int i = 0; i < size; i++)  
        printf("%d ", arr[i]);  
    printf("\n");  
}
```

```
int main() {  
    int arr[] = {12, 11, 13, 5, 6, 7};  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    printf("Unsorted array: \n");  
    printArray(arr, n);  
  
    mergeSort(arr, 0, n - 1);  
  
    printf("Sorted array: \n");  
    printArray(arr, n);  
    return 0;  
}
```

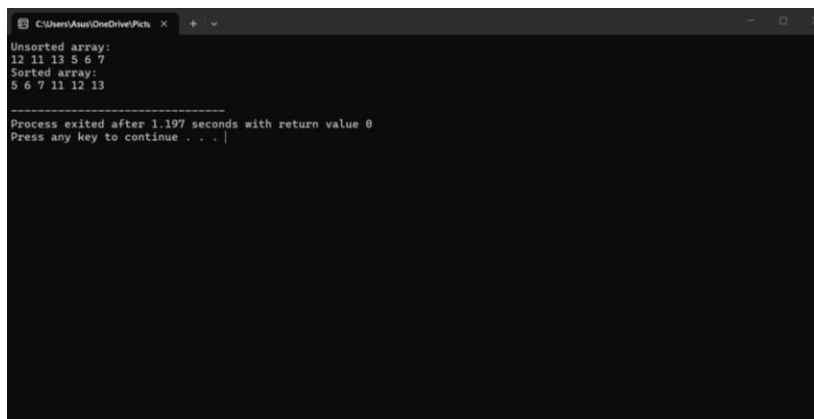
Output:

Unsorted array:

12 11 13 5 6 7

Sorted array:

5 6 7 11 12 13

A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\Aashu\OneDrive\Pics". The window contains the following text: "Unsorted array:", "12 11 13 5 6 7", "Sorted array:", "5 6 7 11 12 13", a separator line "-----", "Process exited after 1.197 seconds with return value 0", and "Press any key to continue . . . |".

```
C:\Users\Aashu\OneDrive\Pics  
Unsorted array:  
12 11 13 5 6 7  
Sorted array:  
5 6 7 11 12 13  
-----  
Process exited after 1.197 seconds with return value 0  
Press any key to continue . . . |
```

Result: Hence the program of merge sort has executed and its output verified successfully

3. BST

Aim:

To implement a Binary Search Tree data structure in C and perform operations like insertion, searching, and inorder traversal.

Description :

A **Binary Search Tree (BST)** is a hierarchical data structure in which each node contains a value and two child nodes (left and right). It follows a specific property:

- The left subtree of a node contains only elements **less than** the node's value.
- The right subtree of a node contains only elements **greater than** the node's value.

This property makes searching, insertion, and deletion operations efficient compared to linear data structures like arrays or linked lists.

BST organizes data in a sorted manner, and an **Inorder Traversal** of a BST always produces elements in ascending order.

Working Principle :

When performing operations in BST:

Insertion :

1. Compare the element with the root.
2. If smaller, move to the left child.
3. If larger, move to the right child.
4. Repeat until a suitable empty position is found.
5. Insert the new node.

Searching :

1. Compare the key with the root.
2. If equal, element is found.
3. If smaller, search in left subtree.
4. If larger, search in right subtree.

Traversal Methods :

- Inorder Traversal (Left → Root → Right)

- Preorder Traversal (Root $\rightarrow$ Left $\rightarrow$ Right)
- Postorder Traversal (Left $\rightarrow$ Right $\rightarrow$ Root)

Example :

Insert the elements:

50, 30, 70, 20, 40, 60, 80

After insertion, the BST structure becomes:

```

      50
     /  \
    30   70
   / \  / \
  20 40 60 80

```

Inorder Traversal Output :

20 30 40 50 60 70 80

This confirms that BST stores elements in sorted order.

If we search for 40:

- Compare with 50 $\rightarrow$ smaller $\rightarrow$ move left
- Compare with 30 $\rightarrow$ larger $\rightarrow$ move right
- Element found

Time Complexity of Binary Search Tree :

Best Case : **$O(\log n)$** (Balanced tree)

Average Case : **$O(\log n)$**

Worst Case : **$O(n)$** (Skewed tree, behaves like linked list.

Space Complexity :

Auxiliary Space : **$O(n)$** (for storing nodes.

Program:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* newNode(int data) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = data;
    node->left = NULL;
    node->right = NULL;
    return node;
}

void inorder(struct Node* root) { if
    (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

struct Node* insert(struct Node* node, int data) { if
    (node == NULL
```

```

return newNode(data);
    }

    if (data < node->data) {
        node->left = insert(node->left, data);
    } else if (data > node->data) {
        node->right = insert(node->right, data);
    }

    return node;
}

struct Node* search(struct Node* root, int data) { if
    (root == NULL || root->data == data) {
        return root;
    }

    if (root->data < data) {
        return search(root->right, data);
    }

    return search(root->left, data);
}

int main() {
    struct Node* root = NULL;
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 20);
    insert(root, 40);

```

```

insert(root, 70);
insert(root, 60);
insert(root, 80);

printf("Inorder traversal of the tree: ");
inorder(root);
printf("\n");

int key = 40;
struct Node* result = search(root, key); if
(result != NULL) {
    printf("Key %d found in BST\n", key);
} else {
    printf("Key %d not found in BST\n", key);
}

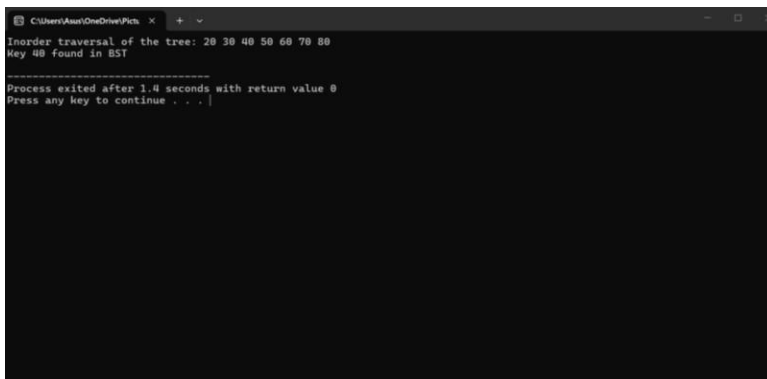
return 0;
}

```

Output:

Inorder traversal of the tree: 20 30 40 50 60 70 80

Key 40 found in BST



```

C:\Users\Aash\OneDrive\Pics
Inorder traversal of the tree: 20 30 40 50 60 70 80
Key 40 found in BST
-----
Process exited after 1.4 seconds with return value 0
Press any key to continue . . . |

```

Result: Hence the program of Binary Search tree has executed and its output verified successfully.

4. Biconnected components

Aim:

To implement an algorithm in C to find the biconnected components of a given undirected graph.

Description :

A **Biconnected Component** is a maximal subgraph of an undirected graph such that the removal of any single vertex does not disconnect the subgraph.

In simple terms, a graph (or subgraph) is said to be **biconnected** if there are at least two different paths between every pair of vertices. This means the graph remains connected even after removing any one vertex.

The algorithm to find biconnected components is based on **Depth First Search (DFS)** and uses two important values:

- **Discovery Time (disc[])** – The time at which a vertex is first visited.
- **Low Value (low[])** – The lowest discovery time reachable from that vertex.

The concept of **Articulation Point** is important here.

Key Concepts :

1. Articulation Point :

A vertex whose removal increases the number of connected components in the graph.

2. Biconnected Graph :

A connected graph with no articulation points.

3. Biconnected Component :

A maximal subgraph that does not contain any articulation point internally.

Working Principle :

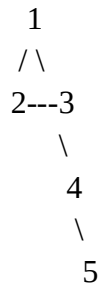
1. Perform DFS traversal of the graph.
2. Maintain discovery time and low value for each vertex.
3. For every DFS edge (u, v):
 - Update low[u] using low[v].
4. If $\text{low}[v] > \text{disc}[u]$, then edge (u, v) forms a separate biconnected component.
5. Continue until all vertices are visited.

Example :

Consider the undirected graph with edges:

(1–2), (2–3), (3–1), (3–4), (4–5)

The graph structure can be visualized as:



In this graph:

- Vertices **1, 2, 3** form a biconnected component (triangle).
- Removing vertex 3 disconnects vertices 4 and 5 from the rest.
- Therefore, vertex 3 is an **articulation point**.
- Edges (3–4) and (4–5) form separate biconnected components.

Thus, the graph contains multiple biconnected components.

Time Complexity Analysis :

Best Case : $O(V + E)$

Average Case : $O(V + E)$

Worst Case : $O(V + E)$

Where:

$V \rightarrow$ Number of vertices

$E \rightarrow$ Number of edges

Since DFS traversal visits each vertex and edge once, the complexity is linear.

Space Complexity :

Auxiliary Space : $O(V)$

Used for storing visited array, discovery time, low values, and recursion stack.

Program:

```
#include<stdio.h>

#include<stdlib.h>

#include <limits.h>


#define MAX_VERTICES 100


int time = 0;
int disc[MAX_VERTICES]; int
low[MAX_VERTICES]; int
parent[MAX_VERTICES]; int
visited[MAX_VERTICES];


void dfs(int u, int* adj[], int V) { visited[u]
    = 1;
    disc[u] = low[u] = ++time;

    for (int v = 0; v < V; v++) { if
        (adj[u][v]) {
            if (!visited[v]) {
                parent[v] = u;
                dfs(v, adj, V);
                low[u] = min(low[u], low[v]);

                if (low[v] > disc[u]) {
                    printf("Biconnected Component: (%d, %d)\n", u, v);
                }
            }
        }
    }
```

```

} else if (v != parent[u]) { low[u] =
min(low[u], disc[v]);
    }
}
}
}

```

```

int min(int a, int b) {
    return (a < b) ? a : b;
}

```

```

void findBiconnectedComponents(int V, int* adj[]) {
    for (int i = 0; i < V; i++) {
        visited[i] = 0;
        parent[i] = -1;
    }

    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            dfs(i, adj, V);
        }
    }
}

```

```

int main() {
    int V = 5;
    int adj[V][V] = {
        {0, 1, 1, 0, 0},
        {1, 0, 1, 0, 0},
        {1, 1, 0, 1, 1},

```

```
{0, 0, 1, 0, 1},  
{0, 0, 1, 1, 0}  
};
```

```
findBiconnectedComponents(V, adj);
```

```
return 0;
```

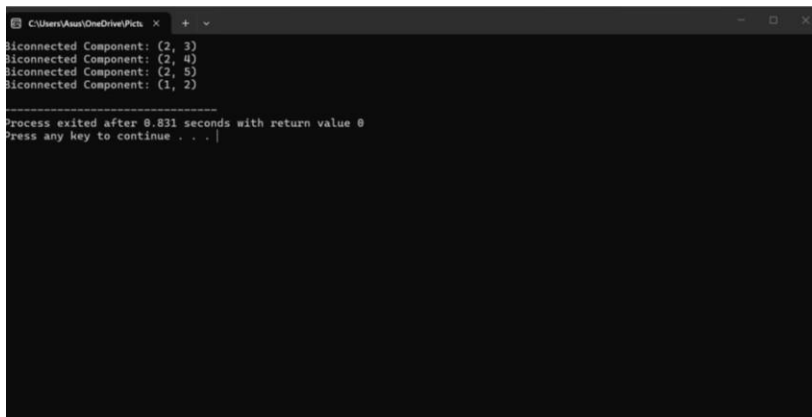
```
}
```

Output (for the given graph):

Biconnected Component: (2, 3)

Biconnected Component: (2, 4)

Biconnected Component: (2, 5)



```
C:\Users\Aashu\OneDrive\Pics > + v  
Biconnected Component: (2, 3)  
Biconnected Component: (2, 4)  
Biconnected Component: (2, 5)  
Biconnected Component: (1, 2)  
-----  
Process exited after 0.831 seconds with return value 0  
Press any key to continue . . . |
```

Result: Hence the program of Biconnected Components has executed and its output verified successfully.

5. Prim's algorithm

Aim:

To implement Prim's algorithm in C to find the Minimum Spanning Tree (MST) of a weighted undirected graph.

Description :

Prim's Algorithm is a **greedy algorithm** used to find the **Minimum Spanning Tree (MST)** of a connected, weighted, undirected graph.

A **Minimum Spanning Tree** is a subset of edges that:

- Connects all vertices of the graph
- Contains no cycles
- Has the minimum possible total edge weight

Prim's Algorithm starts from any one vertex and grows the spanning tree step by step by always selecting the minimum weight edge that connects a vertex inside the tree to a vertex outside the tree.

It maintains two sets of vertices:

- Vertices already included in MST
- Vertices not yet included in MST

At each step, it selects the smallest edge connecting these two sets.

Working Principle :

1. Start with any vertex as the source.
2. Mark it as included in MST.
3. Find the minimum weight edge connecting a visited vertex to an unvisited vertex.
4. Add that edge to the MST.
5. Repeat until all vertices are included.

Prim's algorithm uses a **key array** to store the minimum edge weight required to connect each vertex to the MST.

Example :

Consider the following weighted graph:

Vertices: 0, 1, 2, 3

Edges with weights:

0–1 (2)

0–3 (6)

1–2 (3)

1–3 (8)

2–3 (7)

Starting from vertex 0:

- Select edge 0–1 (2)
- Next minimum edge connecting to new vertex $\rightarrow$ 1–2 (3)
- Next minimum edge $\rightarrow$ 0–3 (6)

The resulting Minimum Spanning Tree contains edges:

0–1 (2)

1–2 (3)

0–3 (6)

Total weight of MST = $2 + 3 + 6 = 11$

Thus, all vertices are connected with minimum total weight.

Time Complexity Analysis of Prim's Algorithm :

Using Adjacency Matrix:

Best Case : $O(V^2)$

Average Case : $O(V^2)$

Worst Case : $O(V^2)$

Where:

$V \rightarrow$ Number of vertices

If implemented using a **Min Heap and Adjacency List**, the time complexity improves to:

$O(E \log V)$

Space Complexity :

Auxiliary Space : $O(V)$

Used for key array, parent array, and visited array.

Program:

```
#include <stdio.h>

#include <limits.h>

#define V 5 // Number of vertices

int minKey(int key[], bool mstSet[]) { int
    min = INT_MAX, min_index;

    for (int v = 0; v < V; v++) {
        if (mstSet[v] == false && key[v] < min) { min =
            key[v];
            min_index = v;
        }
    }

    return min_index;
}

void printMST(int parent[], int graph[V][V]) {
    printf("Edge \tWeight\n");
    for (int i = 1; i < V; i++) {
        printf("%d - %d \t%d \n", parent[i], i, graph[i][parent[i]]);
    }
}
```

```

void primMST(int graph[V][V]) { int
    parent[V];
    int key[V]; bool
    mstSet[V];

    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        mstSet[i] = false;
    }

    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < V - 1; count++) { int
        u = minKey(key, mstSet);
        mstSet[u] = true;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] && mstSet[v] == false && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    printMST(parent, graph);
}

```

```

int main() {

    int graph[V][V] = {

        {0, 2, 0, 6, 0},

        {2, 0, 3, 8, 5},

        {0, 3, 0, 0, 7},

        {6, 8, 0, 0, 9},

        {0, 5, 7, 9, 0}

    };

    primMST(graph);

    return 0;

}

```

Output:

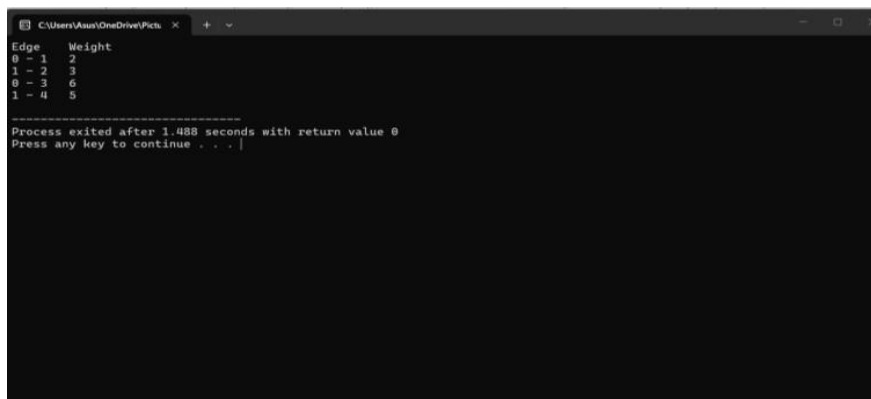
Edge \tWeight

0 - 1 \t2

1 - 2 \t3

0 - 3 \t6

1 - 4 \t5



```

C:\Users\Aussu\OneDrive\Pics
Edge  Weight
0 - 1  2
1 - 2  3
0 - 3  6
1 - 4  5

Process exited after 1.488 seconds with return value 0
Press any key to continue . . .

```

Result: Hence the program Prim's algorithm has executed and its output verified successfully

6. Kruskal's algorithm

Aim:

To implement Kruskal's algorithm in C to find the Minimum Spanning Tree (MST) of a weighted undirected graph.

Description :

Kruskal's Algorithm is a **greedy algorithm** used to find the **Minimum Spanning Tree (MST)** of a connected, weighted, undirected graph.

A **Minimum Spanning Tree** is a spanning tree that:

- Connects all vertices
- Contains exactly $(V - 1)$ edges
- Has no cycles
- Has minimum total edge weight

Unlike Prim's Algorithm, which grows from a single vertex, Kruskal's Algorithm builds the MST by selecting edges in increasing order of their weights.

It uses the concept of **Disjoint Sets (Union-Find)** to detect cycles while adding edges.

Working Principle :

1. Sort all edges in non-decreasing order of their weights.
2. Treat each vertex as a separate set.
3. Pick the smallest edge.
4. Check if it forms a cycle:
 - a. If it does not form a cycle $\rightarrow$ Include it in MST.
 - b. If it forms a cycle $\rightarrow$ Discard it.
5. Repeat until $(V - 1)$ edges are selected.

The Union-Find data structure helps efficiently determine whether two vertices belong to the same set.

Example :

Consider a graph with vertices:

0, 1, 2, 3

Edges with weights:

0-1 (10)

0-2 (6)

0-3 (5)

1-3 (15)

2-3 (4)

Step 1: Sort edges by weight:

2-3 (4)

0-3 (5)

0-2 (6)

0-1 (10)

1-3 (15)

Step 2: Select edges one by one:

- Select 2-3 (4)
- Select 0-3 (5)
- Skip 0-2 (6) (forms cycle)
- Select 0-1 (10)

Now we have $(V - 1 = 3)$ edges.

Final MST edges:

2-3 (4)

0-3 (5)

0-1 (10)

Total Weight = $4 + 5 + 10 = 19$

Time Complexity Analysis of Kruskal's Algorithm :

Best Case : **$O(E \log E)$**

Average Case : **$O(E \log E)$**

Worst Case : **$O(E \log E)$**

Where:

$E \rightarrow$ Number of edges

Most of the time is spent sorting the edges.

Union-Find operations take nearly constant time.

Space Complexity :

Auxiliary Space : **$O(V)$**

Used for parent and rank arrays in Union-Find.

Program:

```
#include <stdio.h>

#include <stdlib.h>

struct Edge {
    int src, dest, weight;
};

struct Graph {
    int V, E;
    struct Edge* edge;
};

struct Graph* createGraph(int V, int E) {
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    graph->V = V;
    graph->E = E;
    graph->edge = (struct Edge*)malloc(graph->E * sizeof(struct Edge));
    return graph;
}

int compare(const void* a, const void* b) {
    struct Edge* a1 = (struct Edge*)a;
    struct Edge* b1 = (struct Edge*)b;
    return a1->weight - b1->weight;
```

```
}
```

```
int find(int parent[], int i) {  
    if (parent[i] == i)  
        return i;  
    return find(parent, parent[i]);  
}
```

```
void union1(int parent[], int rank[], int x, int y) {  
    int xroot = find(parent, x);  
    int yroot = find(parent, y);  
  
    if (rank[xroot] < rank[yroot])  
        parent[xroot] = yroot;  
    else if (rank[xroot] > rank[yroot])  
        parent[yroot] = xroot;  
    else {  
        parent[yroot] = xroot;  
        rank[xroot]++;  
    }  
}
```

```
void KruskalMST(struct Graph* graph) {  
    int V = graph->V;  
    struct Edge result[V];  
    int e = 0;  
    int i = 0;  
  
    qsort(graph->edge, graph->E, sizeof(graph->edge[0]), compare);
```

```

int *parent = (int*)malloc(V * sizeof(int));
int *rank = (int*)malloc(V * sizeof(int));

for (i = 0; i < V; ++i) {
    parent[i] = i;
    rank[i] = 0;
}

i = 0;
while (e < V - 1) {
    struct Edge next_edge = graph->edge[i++];

    int x = find(parent, next_edge.src);
    int y = find(parent, next_edge.dest);

    if (x != y) {
        result[e++] = next_edge;
        union1(parent, rank, x, y);
    }
}

printf("Following are the edges in the constructed MST\n");
for (i = 0; i < e; ++i)
    printf("%d -- %d == %d\n", result[i].src, result[i].dest, result[i].weight);
}

int main() {
    int V = 4;
    int E = 5;
    struct Graph* graph = createGraph(V, E);

```

```
graph->edge[0].src = 0;  
graph->edge[0].dest = 1;  
graph->edge[0].weight = 10;
```

```
graph->edge[1].src = 0;  
graph->edge[1].dest = 2;  
graph->edge[1].weight = 6;
```

```
graph->edge[2].src = 0;  
graph->edge[2].dest = 3;  
graph->edge[2].weight = 5;
```

```
graph->edge[3].src = 1;  
graph->edge[3].dest = 3;  
graph->edge[3].weight = 15;
```

```
graph->edge[4].src = 2;  
graph->edge[4].dest = 3;  
graph->edge[4].weight = 4;
```

```
KruskalMST(graph);
```

```
return 0;
```

```
}
```

Output:

Following are the edges in the constructed MST

0 -- 2 == 6

2 -- 3 == 4

0 -- 3 == 5

```
C:\Users\Asus\OneDrive\Pictu... X + -
Following are the edges in the constructed MST
2 -- 3 == 4
0 -- 3 == 5
0 -- 1 == 10
-----
Process exited after 1.191 seconds with return value 0
Press any key to continue . . . |
```

Result: Hence the program of Kruskal's algorithm has executed and its output verified successfully

7. 0/1 Knapsack problem

Aim:

To implement the 0/1 Knapsack problem using Dynamic Programming in C.

Description :

The **0/1 Knapsack Problem** is a classical optimization problem in Dynamic Programming.

Given:

- N items
- Each item has a **weight** and a **value (profit)**
- A knapsack with maximum capacity **W**

The objective is to select items such that:

- The total weight does not exceed W
- The total profit is maximum

It is called **0/1 Knapsack** because each item can either be:

- Included completely (1)
- Excluded completely (0)

Fractional selection is not allowed.

Dynamic Programming is used to avoid recomputing overlapping subproblems.

Working Principle :

1. Create a DP table of size $(n + 1) \times (W + 1)$.
2. Initialize the first row and first column to 0.
3. For each item:
 - a. If weight of item $\leq$ current capacity:
 - i. Take maximum of:
 1. Including the item
 2. Excluding the item
 - b. Else:

i. Exclude the item.

4. The final answer will be stored in $dp[n][W]$.

Example:

Consider:

Weights:

[10, 20, 30]

Values:

[60, 100, 120]

Knapsack Capacity:

50

Possible combinations:

- Item 1 + Item 2 $\rightarrow$ Weight = 30, Value = 160
- Item 1 + Item 3 $\rightarrow$ Weight = 40, Value = 180
- Item 2 + Item 3 $\rightarrow$ Weight = 50, Value = 220

The best choice is:

Item 2 and Item 3

Maximum Profit = **220**

Thus, the optimal solution gives maximum value without exceeding capacity.

Time Complexity Analysis :

Best Case : **$O(nW)$**

Average Case : **$O(nW)$**

Worst Case : **$O(nW)$**

Where:

$n \rightarrow$ Number of items

$W \rightarrow$ Capacity of knapsack

Every cell in the DP table is computed once.

Space Complexity :

Auxiliary Space : **$O(nW)$**

Space required for DP table.

Program:

```
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int knapSack(int W, int wt[], int val[], int n) {
    int i, w;
    int K[n + 1][W + 1];

    for (i = 0; i <= n; i++) {
        for (w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                K[i][w] = 0;
            else if (wt[i - 1] <= w)
                K[i][w] = max(val[i - 1] + K[i - 1][w - wt[i - 1]], K[i - 1][w]);
            else
                K[i][w] = K[i - 1][w];
        }
    }

    return K[n][W];
}

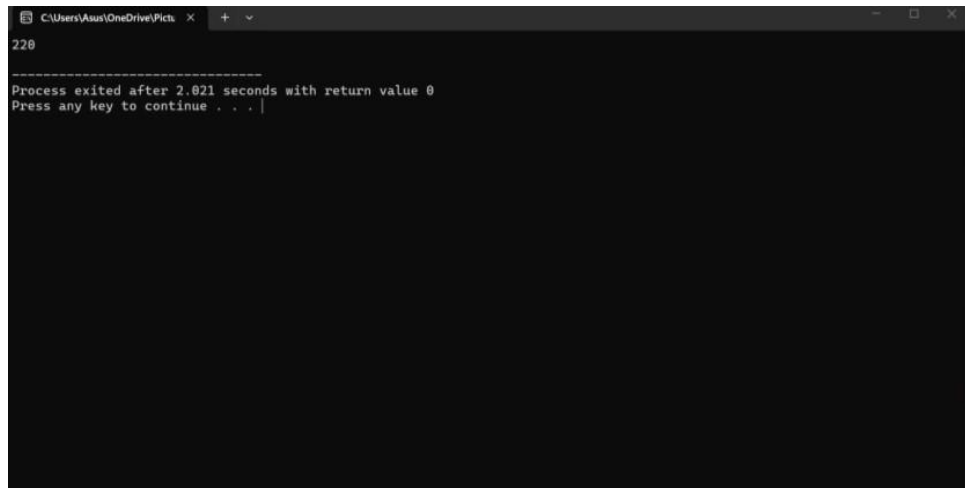
int main()

int val[] = {60, 100, 120};
int wt[] = {10, 20, 30};
int W = 50;
int n = sizeof(val) / sizeof(val[0]);
```

```
printf("%d\n", knapSack(W, wt, val, n));  
return 0;  
}
```

Output:

220



Result: Hence the program of Knapsack problem has executed and its output verified successfully.

8. BFS

Aim:

To implement Breadth-First Search (BFS) algorithm in C to traverse a given graph.

Description :

Breadth First Search (BFS) is a graph traversal algorithm used to explore vertices of a graph in a **level-by-level** manner.

It starts from a selected source vertex and explores all its neighboring vertices first before moving to the next level of neighbors.

BFS uses a **Queue** data structure to maintain the order of traversal. It ensures that vertices are visited in the same order in which they are discovered.

BFS is widely used for finding the shortest path in an unweighted graph and for checking connectivity.

Working Principle :

1. Start from the source vertex.
2. Mark the source as visited.
3. Insert the source into the queue.
4. While the queue is not empty:
 - Remove a vertex from the queue.
 - Visit all unvisited adjacent vertices.
 - Mark them as visited.
 - Insert them into the queue.
5. Repeat until all vertices are visited.

Example :

Consider the graph:

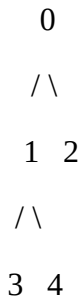
Vertices:

0, 1, 2, 3, 4

Edges:

0–1, 0–2, 1–3, 1–4

Graph representation:



If we start BFS from vertex 0:

Traversal order will be:

0 1 2 3 4

Explanation:

- Visit 0
- Visit neighbors 1 and 2
- Then visit neighbors of 1 $\rightarrow$ 3 and 4

Thus, BFS explores nodes level by level.

Time Complexity :

Best Case : $O(V + E)$

Average Case : $O(V + E)$

Worst Case : $O(V + E)$

Where:

$V \rightarrow$ Number of vertices

$E \rightarrow$ Number of edges

Each vertex and edge is visited only once.

Space Complexity :

Auxiliary Space : $O(V)$

Used for:

- Queue
- Visited array

Program:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX_VERTICES 100
```

```
struct Queue {
```

```
    int front, rear, size;
```

```
    int capacity;
```

```
    int* array;
```

```
};
```

```
struct Queue* createQueue(int capacity) {
```

```
    struct Queue* queue = (struct Queue*)malloc(sizeof(struct Queue));
```

```
    queue->capacity = capacity;
```

```
    queue->front = queue->size = 0;
```

```
    queue->rear = capacity - 1;
```

```
    queue->array = (int*)malloc(queue->capacity * sizeof(int));
```

```
    return queue;
```

```
}
```

```
int isFull(struct Queue* queue) {
```

```
    return (queue->size == queue->capacity);
```

```
}
```

```
int isEmpty(struct Queue* queue) {
```

```
    return (queue->size == 0);
```

```
}
```

```

void enqueue(struct Queue* queue, int item) {
    if (isFull(queue))
        return;
    queue->rear = (queue->rear + 1) % queue->capacity;
    queue->array[queue->rear] = item;
    queue->size = queue->size + 1;
}

```

```

int dequeue(struct Queue* queue) {
    if (isEmpty(queue))
        return INT_MIN;
    int item = queue->array[queue->front];
    queue->front = (queue->front + 1) % queue->capacity;
    queue->size = queue->size - 1;
    return item;
}

```

```

void BFS(int s, int graph[MAX_VERTICES][MAX_VERTICES], int V) {
    int visited[V];
    for (int i = 0; i < V; ++i)
        visited[i] = 0;

    struct Queue* q = createQueue(V);
    enqueue(q, s);
    visited[s] = 1;

    while (!isEmpty(q)) {
        s = dequeue(q);
        printf("%d ", s);
    }
}

```

```

    for (int i = 0; i < V; ++i) {
        if (visited[i] == 0 && graph[s][i] == 1) {
            visited[i] = 1;
            enqueue(q, i);
        }
    }
}

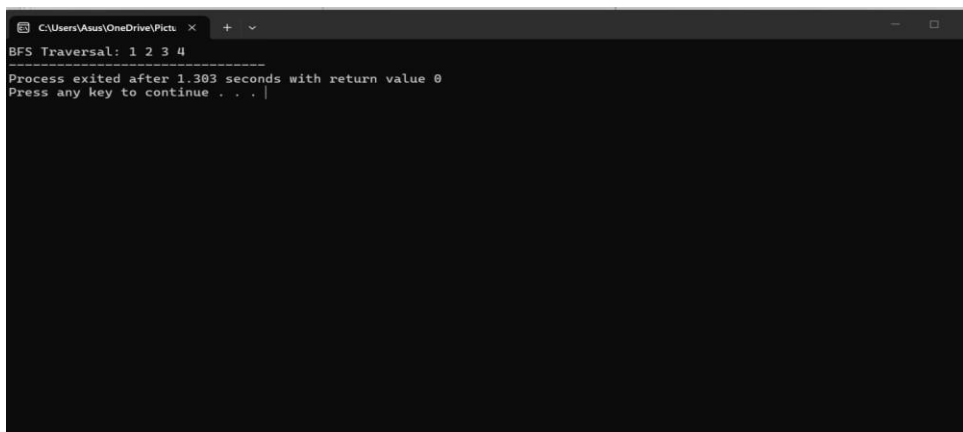
int main() {
    int V = 4;
    int graph[V][V] = {
        {0, 1, 1, 0},
        {1, 0, 1, 1},
        {1, 1, 0, 0},
        {0, 1, 0, 0}
    };
    BFS(0, graph, V);

    return 0;
}

```

Output:

1 2 3



Result: Hence the program of Breadth First Search has executed and its output verified successfully.

9. DFS

Aim:

To implement Depth-First Search (DFS) algorithm in C to traverse a given graph.

Description :

Depth First Search (DFS) is a graph traversal algorithm that explores as far as possible along each branch before backtracking.

Unlike BFS, which uses a queue, DFS uses a **stack** (either explicitly or through recursion). It starts from a source vertex, explores one of its unvisited neighbors, and continues moving deeper until no further unvisited vertex is found. It then backtracks and explores other branches.

DFS naturally forms a **Depth First Spanning Tree** during traversal.

Working Principle :

1. Start from the source vertex.
2. Mark the vertex as visited.
3. Visit one of its unvisited adjacent vertices.
4. Recursively apply DFS to that vertex.
5. If no unvisited neighbors remain, backtrack.
6. Repeat until all vertices are visited.

DFS can be implemented using:

- Recursion (implicit stack)
- Explicit stack data structure

Example :

Consider the graph:

Vertices:

0, 1, 2, 3, 4

Edges:

0–1, 0–2, 1–3, 1–4

Graph representation:

```

    0
  /\
 1  2
 /\
3   4
```

If we start DFS from vertex 0, one possible traversal order is:

0 1 3 4 2

Explanation:

- Start at 0
- Go to 1
- Go deeper to 3
- Backtrack to 1
- Visit 4
- Backtrack to 0
- Visit 2

Thus, DFS explores depth-wise before moving to the next branch.

Time Complexity :

Best Case : $O(V + E)$

Average Case : $O(V + E)$

Worst Case : $O(V + E)$

Where:

$V \rightarrow$ Number of vertices

$E \rightarrow$ Number of edges

Each vertex and edge is visited only once.

Space Complexity :

Recursive Implementation : $O(V)$ (Recursion stack)

Iterative Implementation : $O(V)$ (Explicit stack)

Program:

```
#include <stdio.h>
```

```
#define MAX_VERTICES 100
```

```
void DFSUtil(int v, int visited[], int graph[MAX_VERTICES][MAX_VERTICES], int V) {  
    visited[v] = 1;  
    printf("%d ", v);  
  
    for (int i = 0; i < V; ++i) {  
        if (visited[i] == 0 && graph[v][i] == 1) {  
            DFSUtil(i, visited, graph, V);  
        }  
    }  
}
```

```
void DFS(int V, int graph[MAX_VERTICES][MAX_VERTICES]) {  
    int visited[V];  
    for (int i = 0; i < V; ++i)  
        visited[i] = 0;  
  
    for (int i = 0; i < V; ++i) {  
        if (visited[i] == 0) {  
            DFSUtil(i, visited, graph, V);  
        }  
    }  
}
```

```
}
```

```
int main() {
```

```
    int V = 4;
```

```
    int graph[V][V] = {
```

```
        {0, 1, 1, 0},
```

```
        {1, 0, 1, 1},
```

```
        {1, 1, 0, 0},
```

```
        {0, 1, 0, 0}
```

```
    };
```

```
    printf("Following is Depth First Traversal (starting from vertex 0):\n");
```

```
    DFS(V, graph);
```

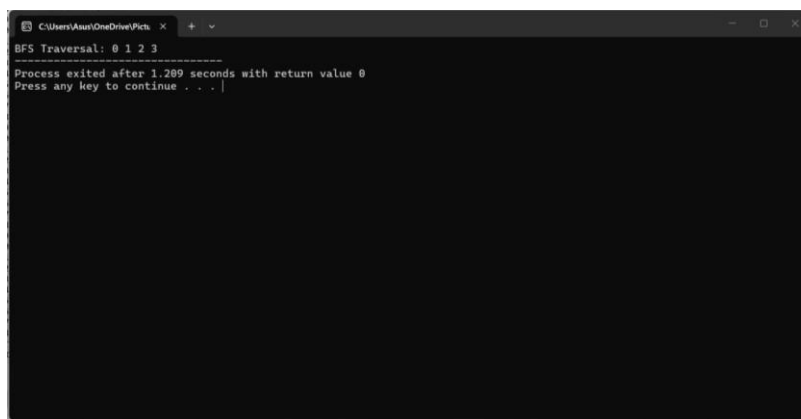
```
    return 0;
```

```
}
```

Output:

Following is Depth First Traversal (starting from vertex 0):

0 1 2 3



Result: Hence the program of Depth First Search has executed and its output verified successfully

10. Topological sort

Aim:

To implement Topological Sort algorithm in C to find a linear ordering of vertices in a directed acyclic graph (DAG).

Description :

Topological Sorting is a linear ordering of vertices in a **Directed Acyclic Graph (DAG)** such that for every directed edge $u \rightarrow v$, vertex u comes before v in the ordering.

Topological sorting is only possible for **Directed Acyclic Graphs**. If the graph contains a cycle, topological ordering is not possible.

It is mainly used in scheduling problems, dependency resolution, and course prerequisite planning.

Topological sorting can be implemented using:

- Depth First Search (DFS)
- Kahn's Algorithm (BFS-based approach using indegree)

In DFS-based implementation, vertices are pushed into a stack after visiting all their adjacent vertices.

Working Principle (DFS Method) :

1. Mark all vertices as unvisited.
2. Perform DFS from each unvisited vertex.
3. After visiting all adjacent vertices of a node, push it onto a stack.
4. After DFS completes, pop elements from the stack to get the topological order.

Example :

Consider the directed graph with vertices:

0, 1, 2, 3, 4, 5

Edges:

$5 \rightarrow 2$

$5 \rightarrow 0$

$4 \rightarrow 0$

$4 \rightarrow 1$

$2 \rightarrow 3$

$3 \rightarrow 1$

Graph structure (dependency view):

5 4

| \ | \

v v v v

2 0 1

|

v

3

|

v

1

One possible Topological Order is:

5 4 2 3 1 0

Explanation:

- 5 comes before 2 and 0
- 4 comes before 0 and 1
- 2 comes before 3

- 3 comes before 1

All dependency conditions are satisfied.

Time Complexity :

Best Case : $O(V + E)$

Average Case : $O(V + E)$

Worst Case : $O(V + E)$

Where:

$V \rightarrow$ Number of vertices

$E \rightarrow$ Number of edges

Each vertex and edge is processed once.

Space Complexity :

Auxiliary Space : $O(V)$

Used for:

- Visited array
- Recursion stack
- Stack to store result

Program:

```
#include <stdio.h>

#include <stdlib.h>

#define MAX_VERTICES 100

void topologicalSortUtil(int v, int visited[], int* stack, int* top, int
adj[MAX_VERTICES][MAX_VERTICES], int V) {

    visited[v] = 1;

    for (int i = 0; i < V; ++i) {
        if (!visited[i] && adj[v][i]) { topologicalSortUtil(i,
            visited, stack, top, adj, V);
        }
    }

    stack[++*top] = v;
}

void topologicalSort(int V, int adj[MAX_VERTICES][MAX_VERTICES]) {
    int* stack = (int*)malloc(V * sizeof(int));
    int visited[V]; int
    i;
    int top = -1;

    for (i = 0; i < V; i++)

        visited[i] = 0;

    for (i = 0; i < V; i++) { if
        (!visited[i])
            topologicalSortUtil(i, visited, stack, &top, adj, V);
    }
```

```

        for (i = top; i >= 0; i--)
            printf("%d ", stack[i]);
    }

int main() { int
    V = 6;
    int adj[V][V] = {
        {0, 1, 1, 0, 0, 0},
        {0, 0, 0, 0, 0, 0},
        {0, 0, 0, 1, 0, 0},
        {0, 0, 0, 0, 0, 1},
        {0, 0, 0, 0, 0, 0},
        {0, 0, 0, 0, 0, 0}
    };

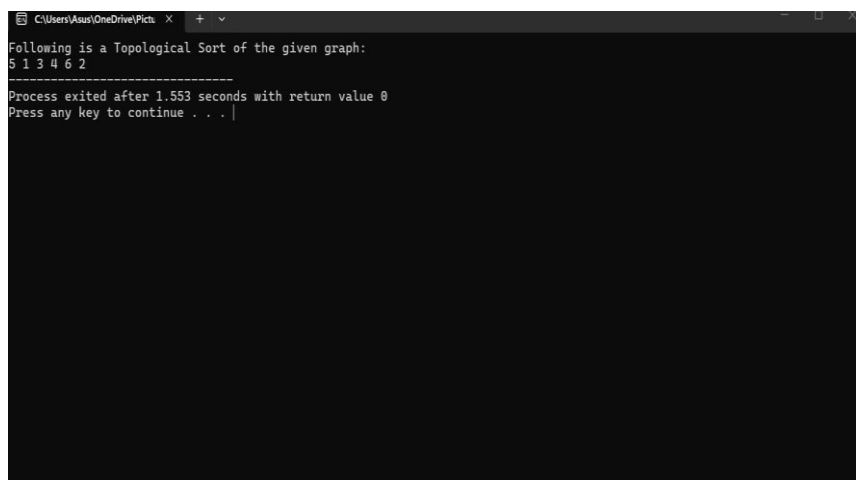
    printf("Following is a Topological Sort of the given graph \n");
    topologicalSort(V, adj);
    return 0;
}

```

Output:

Following is a Topological Sort of the given graph

5 1 3 4 6 2



```

C:\Users\Asus\OneDrive\Pictures
Following is a Topological Sort of the given graph:
5 1 3 4 6 2
-----
Process exited after 1.553 seconds with return value 0
Press any key to continue . . . |

```

Result: Hence the program of Topological Sort has executed and its output verified successfully.

11. KMP

Aim:

To implement the Knuth-Morris-Pratt (KMP) string matching algorithm in C to efficiently find occurrences of a pattern within a text.

Description :

The **Knuth-Morris-Pratt (KMP) Algorithm** is an efficient string matching algorithm used to find occurrences of a **pattern** within a **text**.

Unlike the naive string matching algorithm, KMP avoids unnecessary comparisons by using information from previous matches. It preprocesses the pattern to create an array called the **LPS (Longest Proper Prefix which is also Suffix)** array.

The LPS array helps determine how many characters can be skipped when a mismatch occurs.

KMP is more efficient than the brute-force approach because it does not recheck characters that have already been matched.

Key Concept : LPS Array

LPS (Longest Proper Prefix which is also Suffix):

- Proper prefix → Prefix excluding the full string
- Proper suffix → Suffix excluding the full string

The LPS value at index i indicates the length of the longest prefix which is also a suffix for the substring `pattern[0...i]`.

This helps in deciding how much to shift the pattern when a mismatch occurs.

Working Principle :

KMP works in two main phases:

1. Preprocessing Phase :

- Construct the LPS array for the pattern.
- This helps in skipping unnecessary comparisons.

2. Searching Phase :

- Compare text and pattern characters.
- If characters match → move both pointers.
- If mismatch occurs:
 - Use LPS value to shift the pattern.
 - Do not recompare already matched characters.

Example :

Text:

ABABDABACDABABCABAB

Pattern:

ABABCABAB

Step 1: Construct LPS array for pattern

LPS array becomes:

0 0 1 2 0 1 2 3 4

Step 2: Perform matching

After applying KMP algorithm, the pattern is found at index:

10

So, the pattern "ABABCABAB" occurs in the text starting at position 10.

Time Complexity :

Best Case : $O(n + m)$

Average Case : $O(n + m)$

Worst Case : $O(n + m)$

Where:

$n \rightarrow$ Length of text

$m \rightarrow$ Length of pattern

The algorithm processes each character only once.

Space Complexity :

Auxiliary Space : $O(m)$

Used for storing the LPS array.

Program:

```
#include <stdio.h>

#include <string.h>

void computeLPSArray(char* pat, int M, int* lps) {
    int len = 0;
    lps[0] = 0;

    int i = 1;
    while (i < M) {
        if (pat[i] == pat[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }
}

void KMPSearch(char* pat, char* txt) {
    int M = strlen(pat);
    int N = strlen(txt);

    int lps[M];
    computeLPSArray(pat, M, lps);
```

```

int i = 0;
int j = 0;
while (i < N) {
    if (pat[j] == txt[i]) {
        j++;
        i++;
    }

    if (j == M) {
        printf("Found pattern at index %d \n", i - j);
        j = lps[j - 1];
    }

    else if (i < N && pat[j] != txt[i]) {
        if (j != 0)
            j = lps[j - 1];
        else
            i = i + 1;
    }
}

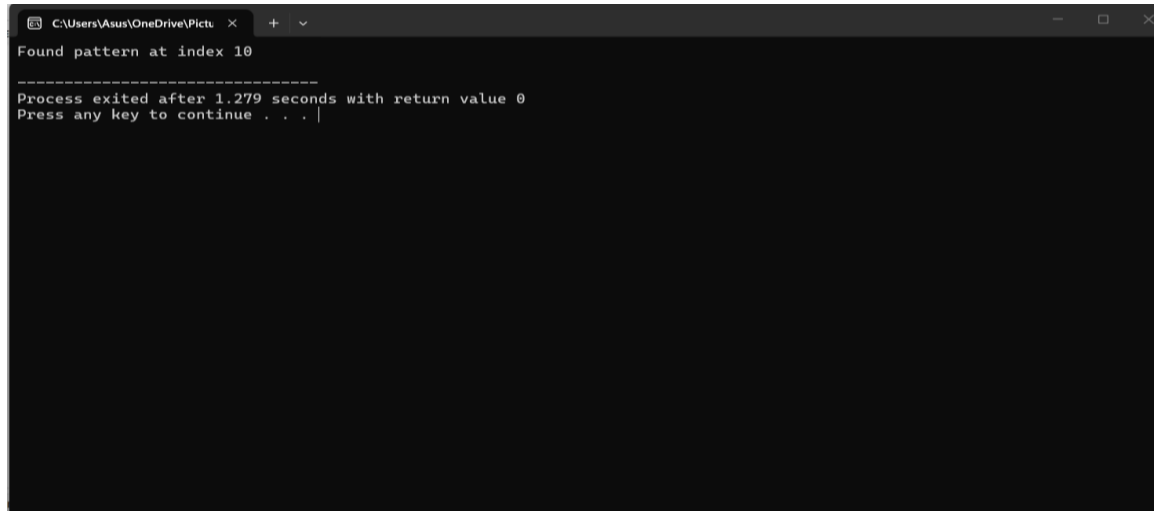
int main() {

    char txt[] = "ABABDABACDABABCABAB";
    char pat[] = "ABABCABAB";
    KMPSearch(pat, txt);
    return 0;
}

```

Output:

Found pattern at index 10



```
C:\Users\Asus\OneDrive\Pictu  x  +  v
Found pattern at index 10
-----
Process exited after 1.279 seconds with return value 0
Press any key to continue . . . |
```

Result: Hence the program of Knuth-Morris-Patt algorithm has executed and its output verified successfully.

12. Rabin - Karp algorithm

Aim:

To implement the Rabin-Karp string matching algorithm in C to efficiently find occurrences of a pattern within a text.

Description :

The **Rabin-Karp Algorithm** is a string matching algorithm that uses **hashing technique** to find a pattern in a given text.

Instead of comparing characters one by one (like Naive algorithm), Rabin-Karp compares the **hash value** of the pattern with the hash value of substrings in the text.

If the hash values match, then a direct character-by-character comparison is performed to confirm the match.

It is especially efficient when searching for multiple patterns in a text.

Working Principle :

1. Compute the hash value of the pattern.
2. Compute the hash value of the first substring of text (same length as pattern).
3. Compare both hash values:
 - If equal → perform character comparison.
4. Slide the pattern over the text:
 - Recalculate hash using **rolling hash technique**.
5. Repeat until the entire text is scanned.

The rolling hash avoids recomputing the hash from scratch.

Rolling Hash Formula :

If:

- t is current hash
- d is number of characters in input alphabet
- q is a prime number
- m is pattern length

Next hash is calculated efficiently using mathematical formula without recomputing whole substring.

Example :

Text:

GEEKS FOR GEEKS

Pattern:

GEEK

Step 1: Compute hash of pattern "GEEK"

Step 2: Compute hash of first substring of text

Compare hashes and slide through text.

The pattern occurs at positions:

0

10

So, the pattern "GEEK" appears twice in the text.

Time Complexity :

Best Case : $O(n + m)$

Average Case : $O(n + m)$

Worst Case : $O(nm)$

Where:

$n \rightarrow$ Length of text

$m \rightarrow$ Length of pattern

Worst case occurs when many hash collisions happen.

Space Complexity :

Auxiliary Space : $O(1)$

Only a few integer variables are used for hashing.

Program:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define d 256
```

```
void search(char* pat, char* txt, int q) {
```

```
int M = strlen(pat);
```

```
int N = strlen(txt);
```

```
int i, j;
```

```
int p = 0;
```

```
int t = 0;
```

```
int h = 1;
```

```
for (i = 0; i < M - 1; i++)
```

$$h = (h * d) \% q;$$

```
for (i = 0; i < M; i++) {
```

$$p = (d * p + \text{pat}[i]) \% q;$$
$$t = (d * t + \text{txt}[i]) \% q;$$
$$\}$$

```
for (i = 0; i <= N - M; i++) {
```

```
if (p == t) {
```

```
for (j = 0; j < M; j++) {
```

```
if (txt[i + j] != pat[j])
```

```
break;
```

$$\}$$

```

        // If p == t and pat[0...M-1] = txt[i, i+1, ...i+M-1]
        if (j == M)
            printf("Pattern found at index %d \n", i);
    }

    // Calculate hash value for next window of text: Remove leading digit, add trailing digit
    if (i < N - M) {
        t = (d * (t - txt[i] * h) + txt[i + M]) % q;

        // We might get negative value of t, converting it
        // to positive
        if (t < 0)
            t = (t + q);
    }
}

int main() {
    char txt[] = "GEEKS FOR GEEKS";
    char pat[] = "GEEK";
    int q = 101; // A prime number

    search(pat, txt, q);

    return 0;
}

```

Output:

Pattern found at index 0

Pattern found at index 10

```
1.25
C:\Users\Aashu\OneDrive\Pics x + -
Pattern found at index 0
Pattern found at index 10
-----
Process exited after 1.25 seconds with return value 0
Press any key to continue . . . |
```

Result: Hence the program of Rabi Karp algorithm has executed and its output verified successfully.

13. Priority Queue using Min or Max heap

Aim:

To Write a C program to implement Priority Queue using Min or Max heap.

Description :

A **Priority Queue** is a special type of queue in which each element is associated with a priority, and elements are served according to their priority rather than their order of insertion.

There are two types of Priority Queues:

- **Min Priority Queue (Min Heap)** – The element with the smallest value has the highest priority.
- **Max Priority Queue (Max Heap)** – The element with the largest value has the highest priority.

A **Heap** is a complete binary tree that satisfies the heap property:

- **Min Heap Property:** Parent node is smaller than or equal to its children.
- **Max Heap Property:** Parent node is greater than or equal to its children.

Heaps are commonly implemented using arrays.

Heap Representation :

For a node at index i in an array:

- Left Child $\rightarrow 2i + 1$
- Right Child $\rightarrow 2i + 2$
- Parent $\rightarrow (i - 1) / 2$

Operations in Priority Queue :

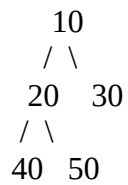
1. **Insertion**
 - Insert the element at the end.
 - Perform heapify-up (bubble up) to maintain heap property.
2. **Deletion (Extract Min / Extract Max)**
 - Remove the root element.
 - Replace it with the last element.
 - Perform heapify-down to maintain heap property.
3. **Peek**
 - Return the root element (highest priority).

Example (Min Heap) :

Insert elements:

40, 20, 30, 10, 50

After building Min Heap:



If we perform **Extract Min**, element removed:

10

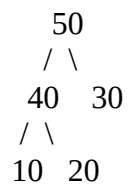
Heap reorganizes to maintain Min Heap property.

Example (Max Heap) :

Insert elements:

40, 20, 30, 10, 50

After building Max Heap:



If we perform **Extract Max**, element removed:

50

Heap reorganizes accordingly.

Time Complexity :

Insertion : **$O(\log n)$**

Deletion : **$O(\log n)$**

Peek : **$O(1)$**

Build Heap : **$O(n)$**

Where:

$n \rightarrow$ Number of elements

Space Complexity :

Auxiliary Space : **$O(n)$**

Used to store heap elements.

Program:

```
#include <stdio.h>

#define MAX 100

int heap[MAX];

int size = 0;

void insert(int value) {
    int i = size++;
    heap[i] = value;

    while (i != 0 && heap[(i - 1) / 2] > heap[i]) {
        int temp = heap[i];
        heap[i] = heap[(i - 1) / 2];
        heap[(i - 1) / 2] = temp;
        i = (i - 1) / 2;
    }
}

int extractMin() {
    if (size <= 0)
        return -1;

    int root = heap[0];
    heap[0] = heap[--size];

    int i = 0;
    while (1) {
        int smallest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;
```

```

    if (left < size && heap[left] < heap[smallest])
        smallest = left;
    if (right < size && heap[right] < heap[smallest])
        smallest = right;

    if (smallest != i) {
        int temp = heap[i];
        heap[i] = heap[smallest];
        heap[smallest] = temp;
        i = smallest;
    } else
        break;
}
return root;
}

```

```

void display() {
    for (int i = 0; i < size; i++)
        printf("%d ", heap[i]);
    printf("\n");
}

```

```

int main() {
    insert(20);
    insert(5);
    insert(15);
    insert(22);
    insert(9);
}

```

```

printf("Priority Queue (Min Heap): ");
display();

printf("Deleted element: %d\n", extractMin());
printf("After deletion: ");
display();

return 0;
}

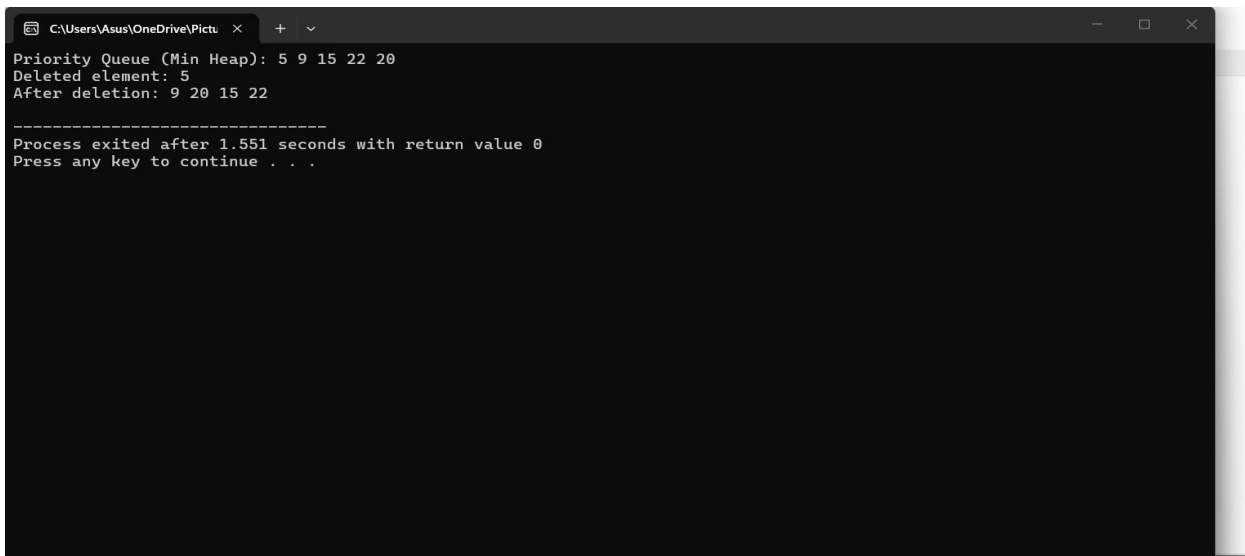
```

OUTPUT:

Priority Queue (Min Heap): 5 9 15 22 20

Deleted element: 5

After Deletion : 9 20 15



```

C:\Users\Asus\OneDrive\Pictu x + v
Priority Queue (Min Heap): 5 9 15 22 20
Deleted element: 5
After deletion: 9 20 15 22

-----
Process exited after 1.551 seconds with return value 0
Press any key to continue . . .

```

Result: Hence the program of Priority Queue using min or map algorithm has executed and its output verified successfully.

14. Shortest path using Dijkstra's algorithm

Aim:

To write a C program to find the shortest path in given graph using Dijkstra's algorithm.

Description :

Dijkstra's Algorithm is a **greedy algorithm** used to find the shortest path from a **single source vertex** to all other vertices in a weighted graph with **non-negative edge weights**.

The algorithm works by selecting the vertex with the smallest tentative distance and updating the distances of its adjacent vertices. It continues this process until all vertices have been visited.

Dijkstra's Algorithm does not work correctly for graphs with negative edge weights.

Working Principle :

1. Assign distance value 0 to the source vertex and infinity to all other vertices.
2. Mark all vertices as unvisited.
3. Select the unvisited vertex with the smallest distance.
4. Mark it as visited.
5. Update the distance of its adjacent vertices if a shorter path is found.
6. Repeat until all vertices are visited.

The algorithm maintains:

- A distance array
- A visited array

Example :

Consider the weighted graph:

Vertices:

0, 1, 2, 3, 4

Edges with weights:

0-1 (10)

0-4 (5)

1-2 (1)

4-1 (3)

4-2 (9)

4-3 (2)

2-3 (4)

3-2 (6)

If we start from source vertex **0**, the shortest distances to all vertices are:

Distance to 0 $\rightarrow$ 0

Distance to 1 $\rightarrow$ 8

Distance to 2 $\rightarrow$ 9

Distance to 3 $\rightarrow$ 7

Distance to 4 $\rightarrow$ 5

Thus, Dijkstra's Algorithm successfully computes the shortest path from source to all other vertices.

Time Complexity :

Using Adjacency Matrix:

Best Case : **$O(V^2)$**

Average Case : **$O(V^2)$**

Worst Case : **$O(V^2)$**

Using Min Heap and Adjacency List:

$O(E \log V)$

Where:

$V \rightarrow$ Number of vertices

$E \rightarrow$ Number of edges

Space Complexity :

Auxiliary Space : **$O(V)$**

Used for:

- Distance array
- Visited array

Program:

```
#include <stdio.h>

#define INF 999
#define MAX 10

int minDistance(int dist[], int visited[], int n)
{
    int min = INF, index;

    for (int i = 0; i < n; i++) {
        if (!visited[i] && dist[i] <= min)
        {
            min = dist[i];
            index = i;
        }
    }
    return index;
}

void dijkstra(int graph[MAX][MAX], int n, int src)
{
    int dist[MAX], visited[MAX];

    for (int i = 0; i < n; i++)
    {
        dist[i] = INF;
        visited[i] = 0;
    }

    dist[src] = 0;

    for (int count = 0; count < n - 1; count++) {
        int u = minDistance(dist, visited, n);
        visited[u] = 1;
```

```

    for (int v = 0; v < n; v++) {
        if (!visited[v] && graph[u][v] &&
            dist[u] != INF &&
            dist[u] + graph[u][v] < dist[v]) {
            dist[v] = dist[u] + graph[u][v];
        }
    }
}

printf("Vertex \t Distance from Source\n");
for (int i = 0; i < n; i++)
    printf("%d \t %d\n", i, dist[i]);
}

int main() {
    int n = 5;
    int graph[MAX][MAX] = {
        {0, 10, 0, 30, 100},
        {10, 0, 50, 0, 0},
        {0, 50, 0, 20, 10},
        {30, 0, 20, 0, 60},
        {100, 0, 10, 60, 0}
    };

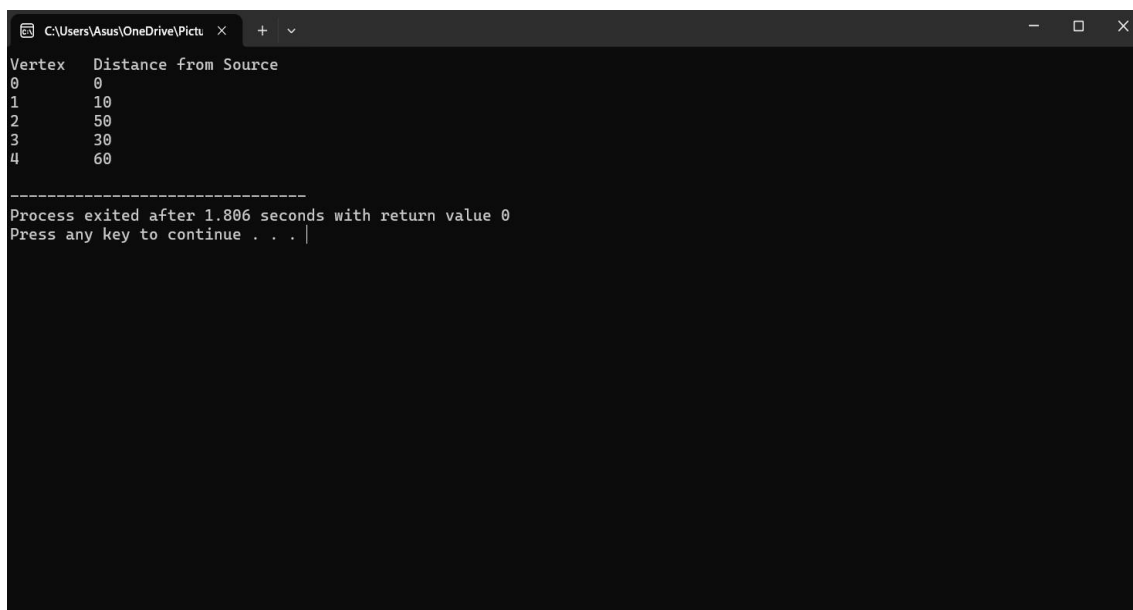
    dijkstra(graph, n, 0);
    return 0;
}

```

OUTPUT:

Vertex Distance from Source

0	0
1	10
2	50
3	30
4	60



```
C:\Users\Asus\OneDrive\Pictu x + v
Vertex Distance from Source
0      0
1      10
2      50
3      30
4      60

-----
Process exited after 1.806 seconds with return value 0
Press any key to continue . . . |
```

Result: Hence the program of Dijkstra's algorithm has executed and its output verified successfully.